Response to "Countdown: Faster Exact Search via Three-Way Joins"

A review prepared with Claude Opus 5.5 in Claude Code

10 October 2026

Abstract

The note under review, dated 10 October 2026 and written with ChatGPT, lowers the leading exponent for deciding whether a Countdown target can be hit exactly from $\frac{2}{3}$ to 0.666223, in expected time $2^{0.666223\, n \log_2 n + O(n)}\,\mathrm{poly}(B)$. We recomputed every numerical inequality it relies on and checked the lemmas we could check without its main source. The arithmetic is right and the reductions we examined are sound. The result is nevertheless much narrower than its title suggests. It rests on a preprint published five days earlier that we could not inspect. It addresses the exact-target question, which our solvers' closest-value task reaches through repeated exact queries: a bounded factor in the game, but not asymptotically. By the note's own Proposition 5.1 it is beaten by the ordinary subset recursion whenever the card values are bounded, as they are in the game. And for the six numbers of the game it changes the bound by a factor of about 1.005.

1 What the note claims

The note treats the number of input numbers n as growing, with their total bit length B a polynomial factor. It splits every expression tree at a centroid node into two subtrees and the *context* above them, a path whose effect on the hole value is a fractional linear map (Lemma 1.1). Inverting the context at the target turns the search into matching three lists, one entry from each, under addition or multiplication. Lemma 2.1 turns multiplicative matching into seven additive instances by rounding logarithms, so any three-list 3SUM algorithm in time $\tilde{O}(K^\tau)$ gives an exact solver in time $2^{\frac{\tau}{\tau+1} n \log_2 n + O(n)}\,\mathrm{poly}(B)$. With $\tau = 2$ this is the earlier $\frac{2}{3}$. Plugging in parameters from a 3SUM algorithm of Alman and Vassilevska Williams [1] gives $\tau = 1.99657$ and the exponent 0.666285. An asymmetric join (Section 3) that uses the actual sizes of the three lists then gives 0.666223.

2 What we checked

The constants. The repository script `paper/check_fast_solver.py` recomputes every numerical condition in Sections 2 and 3. All of them hold.

Condition	Value	Margin
$2\beta + \eta = \alpha$	exact	—
$\gamma > 0.06055$	0.0605550	5.0×10^{-6}
$q < 0.40916$	0.4091519	8.1×10^{-6}
$R > \alpha = 0.05668$	0.0566857	5.7×10^{-6}
$\alpha\gamma > \eta$ and $\beta - \alpha q > \eta$	0.0034320, 0.0034338	2–4 $\times 10^{-6}$
$\tau/(\tau+1) = 199657/299657$	0.6662851	exact
Lemma 3.1: $\gamma > 0.07735$, $q < 0.4549$	0.0773704, 0.4548759	2.0×10^{-5}
Lemma 3.1: $\alpha\gamma > g$, $\alpha(1-2q) > h$	0.0039913, 0.0046543	1–4 $\times 10^{-6}$
Theorem 3.2: $(2-g)/(3-g) < K$	0.66622274	5.7×10^{-7}
Theorem 3.2: $(4-3\delta+h)/(6-4\delta+2h) < K$	0.66621745	5.6×10^{-6}

The margins are small but far above double-precision rounding, so the conditions are not in doubt. The note says an accompanying script checks them with exact rational intervals; that script was not supplied. One condition could not be checked: Lemma 3.1 also needs $R_{23}(\gamma) > \alpha$, and the note does not give the formula for R_{23}.

Lemma 2.1. Correct. If $ab \neq c$ then $ab/c = P/R$ with distinct $P, R \leq 2^{3b}$, so $|\log(ab/c)| \geq 2^{-3b}$. With $Q = 2^{3b+4}$ this gap is at least 16, against a rounding error of at most 3, so the seven shifted additive equalities capture exactly the multiplicative ones.

Theorem 3.2. The case analysis holds, but the case that needs the new asymmetric bound is a sliver. The largest of the three parts, a, must satisfy $K/(2-\delta) < a < 1-K$, that is $0.333684 < a < 0.333777$. Everywhere else the simpler pair or block joins already give K. The whole improvement from 0.666285 to 0.666223 comes from that window.

Proposition 6.2. Sound. Encoding each vector as a product of distinct odd primes makes divisibility $y_b \mid x_a$ equivalent to orthogonality, and the factor 2^K rules out a legal division in the other direction. Scaling both lists by C puts every other one-operation result at or above C.

3 Limitations

The main step rests on a source we could not check. Both new exponents come from Lemmas 36 and 37 and Corollary 31 of [1], an arXiv preprint dated 5 October 2026, five days before the note. We could not inspect it, so we cannot confirm that those results say what the note uses. The note also states without proof that the preprint's algorithm for real numbers can be simulated exactly on b-bit integers. Until both points are confirmed, 0.666223 is a conditional result.

It covers the exact case directly and the closest case by reduction. Our solvers return the value closest to the target, using all four operations, with a fixed canonical answer. When an exact answer exists, as in about 98% of standard rounds, that is exactly the note's problem. Its Proposition 4.1 also returns a canonical exact answer, but under its own order, which ranks the two children of a node by a bit-mask of the cards they use rather than by value. Matching our order would need the same constrained-oracle argument with our ranking key, which the note does not give. When no exact answer exists, the closest one follows from exact queries at $t \pm 1$, $t \pm 2$ and so on, stopping at the first distance that hits; our Erlang and Elixir solvers already search outward like this. In the game the best distance is at most 324, or 918 in cats mode, so this costs at most $2 \cdot 918 + 1$ exact queries, a constant factor on the note's bound. The note's Section 6 treats

closest values separately because, with unbounded targets, the best distance can be exponential in the input length, and then the outward search is no longer polynomial. That is an asymptotic distinction, not a gap for Countdown.

For the real game the simple method wins. Proposition 5.1 shows that when every card is at most a fixed M, which is the case in Countdown ($M = 100$), the full closest-value problem takes $2^{O(n)}\,\mathrm{poly}(B)$ time. That grows more slowly than $2^{0.666\,n\log_2 n}$. The new exponent only matters when card values can be arbitrarily large.

The gain is negligible at six numbers. With $n = 6$, $n\log_2 n = 15.51$, so the change from $\frac{2}{3}$ to 0.666223 alters the leading term by $2^{(2/3-0.666223)\cdot 15.51} \approx 1.005$. The 3SUM algorithms behind it rely on fast matrix multiplication with very large constant factors, and the note says itself that it makes "no claim of practical speedup or research novelty". Our fastest solvers already take about 0.012 ms per round.

No certificates for "no solution". The note says that no separate certificates for negative answers are considered. In our study every unsolvable round has a machine-checked proof of that, through Lean and, for a pilot set, through SAT proofs checked by a verified checker. A faster decision procedure without certificates gives weaker evidence that a round cannot be solved.

Smaller points. The $\frac{2}{3}$ bound it improves on comes from a "previous note" and a companion addendum [3] whose argument is quoted rather than shown. Lemma 3.1 and Theorem 3.2 would be easier to follow if the asymmetric reduction of Chan, Vassilevska Williams and Xu [2] were restated, not only cited by problem number.

4 Conclusion

As far as we could check, the note's mathematics is right: every numerical condition holds and the reductions we examined are sound. Its result is an asymptotic upper bound for the exact-target problem with unbounded card values. It depends on a preprint we could not inspect, and it reaches the closest-value problem that Countdown solvers solve only through repeated exact queries, a bounded overhead in the game. For bounded cards the note itself shows that the subset recursion is asymptotically better, and at six numbers the improvement is about half a percent of the leading term. We would mention it in our paper only as a remark on the theory of the exact-target problem, and would not quote 0.666223 until [1] can be checked.

References

[1] J. Alman and V. Vassilevska Williams. Truly subquadratic 3SUM and truly subcubic APSP via triangles in sparse lopsided graphs. arXiv:2610.06783v1, 5 October 2026. Cited by the note; not inspected.

[2] T. M. Chan, V. Vassilevska Williams and Y. Xu. Hardness for triangle problems under even more believable hypotheses. arXiv:2203.08356v2, 2022.

[3] Countdown: hard bounds and a faster certifying solver. Companion addendum to the note, 10 October 2026.